



UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS
CIENCIAS DE LA COMPUTACIÓN
1ACC0236 - INGENIERÍA DE SOFTWARE 2026-1
EXAMEN PARCIAL

Docente: Carlos Alberto Jara Garcia

Sección: 17915

Hora inicio: 07:00 PM | **Hora fin:** 8:50 PM | **Duración:** 110 min

Indicaciones generales

- Examen **individual** con duración máxima de **1 h 50 min**.
- Descarga el proyecto base **transporte-api**. Ya viene completamente configurado y **NO debes modificar su estructura**: pom.xml, compose.yml, application.yml, entidades JPA (model), excepciones de negocio y arquitectura **package by feature**. Tu trabajo se limita a las capas que se indican en el siguiente bullet.
- Tu trabajo consiste en implementar únicamente los **DTOs** (Request y Response, con las validaciones de Bean Validation que exigen las reglas), los **mappers (MapStruct)** para la conversión entidad ↔ DTO, los **repositorios** (consultas que necesite el servicio), los **servicios** (interfaz + implementación con las reglas de negocio RN1 a RN5) y los **controladores REST** que expongan los endpoints derivados de los criterios de aceptación. **Importante:** las rutas (URLs) de los endpoints deben ser **EXACTAMENTE** las que ya están configuradas en **transporte-api.postman_collection.json**; no debes inventarlas ni modificarlas. **Abre el archivo Postman en el cliente de Postman antes de implementar el controlador** para identificar las rutas, verbos HTTP y bodies que tu API debe atender.
- La base de datos **PostgreSQL** y el cliente web **pgAdmin** se levantan vía Docker. Revisa el **Anexo A** para acceder a pgAdmin (URL y credenciales de inicio de sesión) y el **Anexo B** para registrar el servidor PostgreSQL dentro de pgAdmin.
- En este examen **NO se incluye Spring Security**; todos los endpoints son públicos.
- Para realizar las pruebas de los endpoints implementados utiliza la colección Postman **transporte-api.postman_collection.json** incluida en la raíz del proyecto base.

Instrucciones de entrega

Cada estudiante debe realizar los siguientes pasos 10 min antes de las 08:50 PM

Paso 1: Crear una carpeta utilizando la nomenclatura EA_CODIGOALUMNO. Reemplaza SOLO EL CODIGOALUMNO por tu código de estudiante (sin la letra "U").

ej.: EA_20261234

Paso 2: Colocar los siguientes archivos en la carpeta del Paso 1

- Proyecto biblioteca-api en formato ZIP
- EvidenciaFuncionamiento con las imagenes solicitadas **en formato PPTX**

Paso 3: Comprimir la carpeta en formato **.zip**.

Paso 4: Subir y enviar el archivo **.zip** en la actividad EA1.

Calificación por no presentar el entregable: Si el estudiante no realiza la entrega, obtendrá una calificación de **00 (cero)** en el examen.

Parte I: Caso – TransporteLima

TransporteLima es una empresa de transporte interprovincial con presencia en los terminales de Plaza Norte, Yerbateros y Atocongo. La empresa necesita modernizar la gestión de sus viajes y la venta de boletos. Hasta ahora, el control se lleva en hojas de cálculo compartidas entre los terminales, lo que ha producido inconsistencias frecuentes: viajes con más boletos vendidos que asientos disponibles, pasajeros que aparecen con dos boletos en el mismo viaje, y cancelaciones registradas sin liberar el asiento.

La nueva API REST debe servir como el único punto de verdad sobre la oferta de viajes y los boletos vigentes. La oficina de Operaciones será la primera en consumirla desde su panel administrativo, por lo que la API debe representar fielmente las reglas internas de la empresa, devolver mensajes claros cuando una operación no se pueda completar y mantener los asientos disponibles de cada viaje siempre coherentes con los boletos vendidos.

El público objetivo del panel son los vendedores de los terminales Plaza Norte, Yerbateros y Atocongo, quienes registran nuevos viajes cuando la programación se actualiza, venden boletos a los pasajeros que se acercan al mostrador y procesan las cancelaciones diarias. Necesitan que cada operación falle de manera predecible cuando se rompe una regla, en lugar de aceptar el dato y dejar el problema para después.

Épica

Como vendedor de TransporteLima, quiero administrar la programación de viajes y la venta de boletos a los pasajeros mediante una API REST, para mantener los asientos disponibles siempre consistentes y respetar las reglas internas de la empresa.

User Stories y Criterios de Aceptación

US1 Registro de un nuevo viaje en la programación

Como vendedor, quiero registrar un viaje nuevo en la programación indicando su origen, destino, fecha y hora de salida, tarifa y capacidad, para que el viaje pueda ofertarse a los pasajeros.

Escenario	Dado	Cuando	Entonces
Criterio 1 Escenario de éxito	El vendedor cuenta con los datos completos del viaje (código en formato 4 letras + 4 dígitos, p. ej. LIMA0001; origen y destino; fecha y hora de salida posterior al momento actual; tarifa mayor a cero; capacidad total mayor a cero) y el código aún no existe en la programación.	Registra el viaje en el sistema.	El viaje queda incorporado a la programación con sus asientos disponibles iguales a la capacidad total, y el sistema confirma la operación devolviendo la información del viaje recién creado.
Criterio 2 Escenario de error (datos inválidos o código duplicado)	El vendedor intenta registrar un viaje con datos incompletos, con un código que no cumple el formato 4 letras + 4 dígitos, con fecha-hora de salida en el pasado, con tarifa o capacidad no positivas, o con un código que ya existe en la programación.	Registra el viaje en el sistema.	La operación es rechazada, la programación no se modifica y el sistema responde con un mensaje claro que identifique el motivo del rechazo.

US2 Venta de un boleto a un pasajero

Como vendedor, quiero registrar la venta de un boleto a un pasajero para un viaje específico, indicando su DNI y el número de asiento elegido, para que el sistema controle automáticamente la disponibilidad y las reglas de la empresa.

Escenario	Dado	Cuando	Entonces
Criterio 1 Escenario de éxito	El vendedor cuenta con los datos válidos del boleto (viaje existente con al menos un asiento disponible, DNI del pasajero con 8 dígitos, número de asiento dentro de la capacidad del viaje) y el pasajero todavía no tiene un boleto activo para ese mismo viaje.	Registra el boleto en el sistema.	El boleto queda registrado en estado activo, los asientos disponibles del viaje disminuyen en una unidad y el sistema confirma la operación devolviendo los datos del boleto junto con el código y la fecha-hora del viaje asociado.
Criterio 2 Escenario de error (regla de negocio incumplida)	El vendedor intenta registrar un boleto que infringe una regla de negocio (viaje inexistente, viaje sin asientos disponibles, o pasajero que ya tiene un boleto activo para el mismo viaje).	Registra el boleto en el sistema.	La operación es rechazada, los asientos del viaje no se modifican y el sistema responde con un mensaje que indique cuál regla se incumplió.

US3 Cancelación de un boleto

Como vendedor, quiero registrar la cancelación de un boleto activo, para que el asiento vuelva a estar disponible y el pasajero recupere su cupo en el viaje.

Escenario	Dado	Cuando	Entonces
Criterio 1 Escenario de éxito	El vendedor identifica un boleto existente en estado ACTIVO.	Registra la cancelación del boleto.	El boleto queda marcado como CANCELADO, los asientos disponibles del viaje aumentan en una unidad y el sistema confirma la operación devolviendo el boleto actualizado.
Criterio 2 Escenario de error (cancelación inválida)	El vendedor intenta cancelar un boleto que no existe o que ya fue cancelado previamente.	Registra la cancelación del boleto.	La operación es rechazada, los asientos del viaje no se modifican y el sistema responde con un mensaje que indique la causa (no encontrado o ya cancelado).

Parte II: Reglas de negocio

Las reglas de negocio que deben respetarse en la capa de servicio son las siguientes. El sistema debe rechazar la operación y responder con un mensaje claro cuando alguna se incumpla.

Código	Regla
RN1	Código de viaje único en la programación. El código identifica de manera única a cada viaje. Al registrar un viaje nuevo, si el código ingresado ya pertenece a otro viaje de la programación, el registro no es válido y debe rechazarse.
RN2	Asientos requeridos para vender. Un boleto solo es válido cuando el viaje tiene al menos un asiento disponible en el momento del registro. Si el viaje no tiene asientos disponibles, la venta debe rechazarse.
RN3	Un boleto activo por pasajero y viaje. Un pasajero (DNI de 8 dígitos) no puede tener más de un boleto en estado ACTIVO para el mismo viaje. Si ya cuenta con un boleto activo en ese viaje, no se le puede vender otro hasta que cancele el primero.
RN4	Consistencia de los asientos disponibles. Los asientos disponibles de cada viaje deben reflejar siempre los boletos en curso: al registrarse una venta válida, el viaje queda con un asiento menos disponible; al registrarse la cancelación de ese boleto, el asiento vuelve a estar disponible.
RN5	Cancelación única por boleto. Cada boleto solo puede pasar al estado CANCELADO una vez. Si el boleto ya se encuentra en estado CANCELADO, una segunda cancelación sobre el mismo boleto debe rechazarse.

Rúbrica de evaluación (20 pts)

El detalle por niveles de desempeño se encuentra en el archivo Rúbrica_EA.xlsx. Resumen de criterios:

N°	Criterio	Descripción	Puntaje
1	Implementación de DTOs, Mapper (MapStruct), Repository, Service y Controller	DTOs Request/Response con las validaciones de Bean Validation que exigen las reglas (RN), Mapper con MapStruct para la conversión entidad ↔ DTO, Repository con las consultas necesarias para el servicio (p. ej. búsqueda por código de viaje, búsqueda de boleto activo por pasajero y viaje), Service (interfaz + implementación) que ejecuta los flujos de cada user story usando @Transactional y reutilizando las excepciones de negocio ya provistas en el proyecto (viaje/exception, boleto/exception, shared/exception), y Controller REST que expone los endpoints con las rutas y verbos definidos en la colección Postman, devolviendo códigos de estado de éxito coherentes con cada operación (201 al crear, 200 al consultar o actualizar).	5 pts
2	Cumplimiento de reglas de negocio (RN1 a RN5)	El servicio aplica correctamente las cinco reglas (RN1 a RN5), mantiene los asientos disponibles consistentes luego de cada venta y cancelación, y lanza la excepción de negocio apropiada (de las ya provistas en el proyecto base) cuando una regla se incumple.	6 pts
3	Evidencia de funcionamiento	Para obtener las evidencias debes utilizar el archivo Postman transporte-api.postman_collection.json que se te ha brindado en la raíz del proyecto base. Se entrega la colección Postman cubriendo los 6 escenarios (éxito + alternativo de US1, US2 y US3) ejecutándose contra la API del Anexo A, y se entregan en el PPT (a) capturas con la VENTANA COMPLETA de Postman mostrando request body y response body sin recortes, y (b) capturas desde pgAdmin (Anexo A/B) del contenido de las tablas viajes y boletos para evidenciar que los datos quedaron persistidos. DEBE FIGURAR SU CAMARA PRENDIDA (SE EVALUA EN RUBRICA)	9 pts
		TOTAL	20 pts

Anexo A – Acceso a pgAdmin (web)

Junto con PostgreSQL, el archivo compose.yml levanta también un contenedor de pgAdmin en su versión web. Sigue estos pasos:

1. Iniciar PostgreSQL y pgAdmin (Docker)

```
docker compose up -d
```

2. Verificar que los contenedores están corriendo

```
docker compose ps
```

3. Abrir pgAdmin en el navegador

<http://localhost:5070>

4. Iniciar sesión con las siguientes credenciales

Campo	Valor
Correo electrónico	admin@transporte.com
Contraseña	admin123

Estos valores ya están configurados en el servicio pgAdmin definido en compose.yml. No es necesario modificarlos.

5. Detener los contenedores al finalizar el examen

```
docker compose down
```

Anexo B – Registrar el servidor PostgreSQL en pgAdmin

Una vez dentro de pgAdmin (Anexo A), registra el servidor PostgreSQL para poder explorar la base de datos. En la barra lateral haz clic derecho sobre Servers → Register → Server... y completa las pestañas General y Connection con los datos de la siguiente tabla.

Campo	Valor
Name (pestaña General)	TransporteLima
Host (pestaña Connection)	postgres
Port	5432
Maintenance database	transportedb
Username	transporte
Password	transporte123

Importante: como pgAdmin corre dentro de Docker en la misma red que PostgreSQL, el Host es el nombre del servicio (postgres) y el Port es 5432, no el puerto del host (5090). El puerto 5090 solo aplica si te conectas desde fuera de Docker (p. ej. un cliente SQL en tu máquina).

Estos mismos datos ya están configurados en src/main/resources/application.yml para la conexión de la API. No es necesario modificarlos.

Nota sobre los puertos

Si los puertos 5090 (PostgreSQL) o 5070 (pgAdmin) están ocupados en tu máquina, debes modificarlos en DOS archivos del proyecto base. En los ejemplos usaremos 5091 para PostgreSQL y 5071 para pgAdmin, pero puedes elegir cualquier puerto libre de tu máquina.

1) En el archivo compose.yml de la raíz del proyecto, ubica la sección ports: del servicio correspondiente y cambia el primer número (el del host). Por ejemplo, para PostgreSQL la línea "5090:5432" pásala a "5091:5432"; para pgAdmin la línea "5070:80" pásala a "5071:80". El segundo número (después de los dos puntos) NO se modifica.

2) Si modificas el puerto de PostgreSQL (ejemplo 5091), abre el archivo src/main/resources/application.yml y actualiza el puerto dentro de la propiedad spring.datasource.url (jdbc:postgresql://localhost:5091/transportedb).

Después de los cambios, ejecuta docker compose down y docker compose up -d para que las nuevas configuraciones se apliquen.